

Relatorio do trial

PARTICIPANTE

Luis Henrique Gonçalves Barbosa

KATA

Tarifador de Estacionamento Rotativo

TRATAMENTO

Sem assistente de IA

TEMPO FINAL

23:45

TEMPO ESGOTADO

Nao

INICIO

16/09/2026, 13:11:59

FIM

16/09/2026, 13:35:44

RESULTADO E OBSERVACOES

8 de 8 testes unitários de aceitação passaram com sucesso (100%). Codificação manual.

ARQUIVOS ANEXADOS

3 arquivo(s) de código ou texto anexado(s) ao trial.

- texto-do-trial.txt (165 linhas)
- TarifadorEstacionamento.java (135 linhas)
- TipoVeiculo.java (18 linhas)

texto-do-trial.txt

Participante: Luis Henrique Gonçalves Barbosa
Kata: Tarifador de Estacionamento Rotativo
Tratamento: SEM_IA
Tempo: 23:45

Código implementado:

```
// === TarifadorEstacionamento.java ===
package br.ufc.lab02.estacionamento;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Duration;
import java.time.LocalDateTime;

public class TarifadorEstacionamento {

    private static final long MINUTOS_FRANQUIA = 15;
    private static final long MINUTOS_PRIMEIRA_HORA = 60;
    private static final long MINUTOS_POR_HORA = 60;
    private static final long MINUTOS_POR_DIA = 24 * MINUTOS_POR_HORA;
    private static final BigDecimal TARIFA_PRIMEIRA_HORA = BigDecimal.valueOf(12.00);
    private static final BigDecimal TARIFA_HORA_ADICIONAL = BigDecimal.valueOf(6.00);
    private static final BigDecimal TETO_DIARIA = BigDecimal.valueOf(60.00);
    private static final BigDecimal FATOR_DESCONTO_CONVENIO = BigDecimal.valueOf(0.85);

    public BigDecimal calcularTarifa(
        LocalDateTime entrada,
        LocalDateTime saida,
        TipoVeiculo tipo,
        boolean possuiConvenio) {

        validarDadosDaPermanencia(entrada, saida, tipo);

        long minutosTotais = calcularMinutosTotais(entrada, saida);
        if (estaDentroDaFranquia(minutosTotais)) {
            return formatarValor(BigDecimal.ZERO);
        }

        BigDecimal valorDaPermanencia = calcularValorDaPermanencia(minutosTotais);
        BigDecimal valorComTipoDeVeiculo = aplicarFatorDoVeiculo(valorDaPermanencia,
tipo);
        BigDecimal valorComConvenio = aplicarConvenio(valorComTipoDeVeiculo,
possuiConvenio);

        return formatarValor(valorComConvenio);
    }

    private void validarDadosDaPermanencia(
        LocalDateTime entrada,
        LocalDateTime saida,
        TipoVeiculo tipo) {

        if (entrada == null) {
            throw new IllegalArgumentException("Entrada e saida nao podem ser nulas.");
        }

        if (saida == null) {

```

```

        throw new IllegalArgumentException("Entrada e saida nao podem ser nulas.");
    }

    if (saida.isBefore(entrada)) {
        throw new IllegalArgumentException("Saida nao pode ser anterior a entrada.");
    }

    if (tipo == null) {
        throw new IllegalArgumentException("Tipo de veiculo obrigatorio.");
    }
}

private long calcularMinutosTotais(LocalDateTime entrada, LocalDateTime saida) {
    Duration tempoDePermanencia = Duration.between(entrada, saida);
    return tempoDePermanencia.toMinutes();
}

private boolean estaDentroDaFranquia(long minutosTotais) {
    return minutosTotais <= MINUTOS_FRANQUIA;
}

private BigDecimal calcularValorDaPermanencia(long minutosTotais) {
    long diasCompletos = minutosTotais / MINUTOS_POR_DIA;
    long minutosRestantes = minutosTotais % MINUTOS_POR_DIA;

    BigDecimal valorDosDiasCompletos = calcularValorDosDiasCompletos(diasCompletos);
    BigDecimal valorDoPeriodoRestante =
calcularValorDoPeriodoRestante(minutosRestantes);

    return valorDosDiasCompletos.add(valorDoPeriodoRestante);
}

private BigDecimal calcularValorDosDiasCompletos(long diasCompletos) {
    BigDecimal quantidadeDeDias = BigDecimal.valueOf(diasCompletos);
    return TETO_DIARIA.multiply(quantidadeDeDias);
}

private BigDecimal calcularValorDoPeriodoRestante(long minutosRestantes) {
    if (minutosRestantes == 0) {
        return BigDecimal.ZERO;
    }

    if (minutosRestantes <= MINUTOS_PRIMEIRA_HORA) {
        return TARIFA_PRIMEIRA_HORA;
    }

    long minutosAdicionais = minutosRestantes - MINUTOS_PRIMEIRA_HORA;
    long horasAdicionais = arredondarHorasParaCima(minutosAdicionais);
    BigDecimal quantidadeDeHoras = BigDecimal.valueOf(horasAdicionais);
    BigDecimal custoAdicional = TARIFA_HORA_ADICIONAL.multiply(quantidadeDeHoras);
    BigDecimal valorCalculado = TARIFA_PRIMEIRA_HORA.add(custoAdicional);

    if (valorCalculado.compareTo(TETO_DIARIA) > 0) {
        return TETO_DIARIA;
    }

    return valorCalculado;
}

private long arredondarHorasParaCima(long minutos) {
    long horasCompletas = minutos / MINUTOS_POR_HORA;

```

```

        long minutosDaFracao = minutos % MINUTOS_POR_HORA;

        if (minutosDaFracao > 0) {
            return horasCompletas + 1;
        }

        return horasCompletas;
    }

    private BigDecimal aplicarFatorDoVeiculo(BigDecimal valor, TipoVeiculo tipo) {
        BigDecimal fatorDoVeiculo = BigDecimal.valueOf(tipo.getFatorMultiplicador());
        return valor.multiply(fatorDoVeiculo);
    }

    private BigDecimal aplicarConvenio(BigDecimal valor, boolean possuiConvenio) {
        if (possuiConvenio) {
            return valor.multiply(FATOR_DESCONTO_CONVENIO);
        }

        return valor;
    }

    private BigDecimal formatarValor(BigDecimal valor) {
        return valor.setScale(2, RoundingMode.HALF_EVEN);
    }
}

```

```

// === TipoVeiculo.java ===
package br.ufc.lab02.estacionamento;

public enum TipoVeiculo {
    MOTO(0.50),
    CARRO_PASSEIO(1.00),
    CAMINHONETE(1.25);

    private final double fatorMultiplicador;

    TipoVeiculo(double fatorMultiplicador) {
        this.fatorMultiplicador = fatorMultiplicador;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }
}

```

TarifadorEstacionamento.java

```
package br.ufc.lab02.estacionamento;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Duration;
import java.time.LocalDateTime;

public class TarifadorEstacionamento {

    private static final long MINUTOS_FRANQUIA = 15;
    private static final long MINUTOS_PRIMEIRA_HORA = 60;
    private static final long MINUTOS_POR_HORA = 60;
    private static final long MINUTOS_POR_DIA = 24 * MINUTOS_POR_HORA;
    private static final BigDecimal TARIFA_PRIMEIRA_HORA = BigDecimal.valueOf(12.00);
    private static final BigDecimal TARIFA_HORA_ADICIONAL = BigDecimal.valueOf(6.00);
    private static final BigDecimal TETO_DIARIA = BigDecimal.valueOf(60.00);
    private static final BigDecimal FATOR_DESCONTO_CONVENIO = BigDecimal.valueOf(0.85);

    public BigDecimal calcularTarifa(
        LocalDateTime entrada,
        LocalDateTime saida,
        TipoVeiculo tipo,
        boolean possuiConvenio) {

        validarDadosDaPermanencia(entrada, saida, tipo);

        long minutosTotais = calcularMinutosTotais(entrada, saida);
        if (estaDentroDaFranquia(minutosTotais)) {
            return formatarValor(BigDecimal.ZERO);
        }

        BigDecimal valorDaPermanencia = calcularValorDaPermanencia(minutosTotais);
        BigDecimal valorComTipoDeVeiculo = aplicarFatorDoVeiculo(valorDaPermanencia,
            tipo);
        BigDecimal valorComConvenio = aplicarConvenio(valorComTipoDeVeiculo,
            possuiConvenio);

        return formatarValor(valorComConvenio);
    }

    private void validarDadosDaPermanencia(
        LocalDateTime entrada,
        LocalDateTime saida,
        TipoVeiculo tipo) {

        if (entrada == null) {
            throw new IllegalArgumentException("Entrada e saida nao podem ser nulas.");
        }

        if (saida == null) {
            throw new IllegalArgumentException("Entrada e saida nao podem ser nulas.");
        }

        if (saida.isBefore(entrada)) {
            throw new IllegalArgumentException("Saida nao pode ser anterior a entrada.");
        }

        if (tipo == null) {
            throw new IllegalArgumentException("Tipo de veiculo nao pode ser nulo.");
        }
    }
}
```

```

        throw new IllegalArgumentException("Tipo de veiculo obrigatorio.");
    }
}

private long calcularMinutosTotais(LocalDateTime entrada, LocalDateTime saida) {
    Duration tempoDePermanencia = Duration.between(entrada, saida);
    return tempoDePermanencia.toMinutes();
}

private boolean estaDentroDaFranquia(long minutosTotais) {
    return minutosTotais <= MINUTOS_FRANQUIA;
}

private BigDecimal calcularValorDaPermanencia(long minutosTotais) {
    long diasCompleto = minutosTotais / MINUTOS_POR_DIA;
    long minutosRestantes = minutosTotais % MINUTOS_POR_DIA;

    BigDecimal valorDosDiasCompleto = calcularValorDosDiasCompleto(diasCompleto);
    BigDecimal valorDoPeriodoRestante =
calcularValorDoPeriodoRestante(minutosRestantes);

    return valorDosDiasCompleto.add(valorDoPeriodoRestante);
}

private BigDecimal calcularValorDosDiasCompleto(long diasCompleto) {
    BigDecimal quantidadeDeDias = BigDecimal.valueOf(diasCompleto);
    return TETO_DIARIA.multiply(quantidadeDeDias);
}

private BigDecimal calcularValorDoPeriodoRestante(long minutosRestantes) {
    if (minutosRestantes == 0) {
        return BigDecimal.ZERO;
    }

    if (minutosRestantes <= MINUTOS_PRIMEIRA_HORA) {
        return TARIFA_PRIMEIRA_HORA;
    }

    long minutosAdicionais = minutosRestantes - MINUTOS_PRIMEIRA_HORA;
    long horasAdicionais = arredondarHorasParaCima(minutosAdicionais);
    BigDecimal quantidadeDeHoras = BigDecimal.valueOf(horasAdicionais);
    BigDecimal custoAdicional = TARIFA_HORA_ADICIONAL.multiply(quantidadeDeHoras);
    BigDecimal valorCalculado = TARIFA_PRIMEIRA_HORA.add(custoAdicional);

    if (valorCalculado.compareTo(TETO_DIARIA) > 0) {
        return TETO_DIARIA;
    }

    return valorCalculado;
}

private long arredondarHorasParaCima(long minutos) {
    long horasCompleto = minutos / MINUTOS_POR_HORA;
    long minutosDaFracao = minutos % MINUTOS_POR_HORA;

    if (minutosDaFracao > 0) {
        return horasCompleto + 1;
    }

    return horasCompleto;
}

```

```
private BigDecimal aplicarFatorDoVeiculo(BigDecimal valor, TipoVeiculo tipo) {
    BigDecimal fatorDoVeiculo = BigDecimal.valueOf(tipo.getFatorMultiplicador());
    return valor.multiply(fatorDoVeiculo);
}

private BigDecimal aplicarConvenio(BigDecimal valor, boolean possuiConvenio) {
    if (possuiConvenio) {
        return valor.multiply(FATOR_DESCONTO_CONVENIO);
    }

    return valor;
}

private BigDecimal formatarValor(BigDecimal valor) {
    return valor.setScale(2, RoundingMode.HALF_EVEN);
}
}
```

TipoVeiculo.java

```
package br.ufc.lab02.estacionamento;

public enum TipoVeiculo {
    MOTO(0.50),
    CARRO_PASSEIO(1.00),
    CAMINHONETE(1.25);

    private final double fatorMultiplicador;

    TipoVeiculo(double fatorMultiplicador) {
        this.fatorMultiplicador = fatorMultiplicador;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }
}
```